8주차 풀이 읽기자료

해설 사용 순서

먼저 학생 실습에서 문제를 독립적으로 풉니다. 이 자료는 힌트와 완전 해설을 포함합니다. 필요한 단계까지만 읽고 다시 풀이합니다.

표시된 예시는 강의에서 만든 예시입니다. 원문과 전체 제약은 각 공식 링크에서 확인합니다.

학생 실습

교사용 진행안

8주차 코드 내려받기

55 · 단계별 힌트

LeetCode 55: Jump Game · Medium

H1. 지금까지 도달 가능한 가장 먼 곳은 어디인가요?

H2. 한 칸도 건널 수 없는 빈틈을 뒤쪽 값이 고칠 수 있나요?

H3. 도달 가능한 index에서만 farthest=max(farthest,index+step)을 갱신합니다.

55 · 정당성과 비용

이전 도달 집합은 0..farthest의 연속 구간입니다. 도달한 위치에서 최대 step 이내의 모든 거리가 허용되어 같은 형태를 유지합니다. index>farthest면 건널 방법이 없습니다.

n=길이. O(n) 시간·O(1) 보조 공간.

55 · 반례와 검증

[2,3,0,0,0]에서 처음부터 최대 거리로 점프하면 위치 2의 0에서 멈춥니다. 도달 가능한 위치 1도 탐색하면 마지막 위치까지 갈 수 있습니다. 알고리즘은 가능한 위치들의 도달 범위를 유지합니다.

```
nums=[2,0,1,0] → True
```

실행 전에 상태와 반환값을 적고 위의 경계 사례도 확인합니다.

55 · 구현

```
def can_jump(nums):
    farthest = 0
    for index, step in enumerate(nums):
        if index > farthest:
            return False
        farthest = max(farthest, index + step)
    return True
```

45 · 단계별 힌트

LeetCode 45: Jump Game II · Medium

H1. 현재 점프 수로 갈 수 있는 구간의 끝은 어디인가요?

H2. 다음 점프를 확정하기 전에 어느 위치들을 봐야 하나요?

H3. end는 현재 점프 수의 끝, farthest는 다음 범위입니다. index==end일 때만 jumps를 늘립니다.

45 · 정당성과 비용

현재 레벨의 모든 도달 위치를 스캔한 뒤 다음 최대 범위를 확정합니다. 이는 암묵적 무가중 그래프의 BFS 레벨 확장이므로 더 적은 점프로 끝을 넘을 수 없습니다.

n=길이. O(n) 시간·O(1) 보조 공간.

45 · 반례와 검증

목표는 n-1입니다. 마지막 위치까지 반복하면 점프를 한 번 더 셉니다. [0]의 답은 0입니다. 비도달 입력은 공식 보장 밖입니다.

```
nums=[2,3,0,1,0] → 2
```

실행 전에 상태와 반환값을 적고 위의 경계 사례도 확인합니다.

45 · 구현

```
def min_jumps(nums):
    end = farthest = jumps = 0
    for index in range(len(nums) - 1):
        farthest = max(farthest, index + nums[index])
        if index == end:
            jumps += 1
            end = farthest
    return jumps
```

134 · 단계별 힌트

LeetCode 134: Gas Station · Medium

H1. 전체 연료 부족이면 시작점이 도움이 되나요?

H2. s에서 i까지 처음 실패하면 그 사이에서 시작할 수 있나요?

H3. 차이의 total과 후보 구간 tank를 합산합니다. tank<0이면 i+1부터 다시 시작합니다.

134 · 정당성과 비용

s..i에서 처음 실패했다면 s..j-1의 누적합은 음수가 아닙니다. 따라서 중간 j에서 시작한 j..i 합은 실패 합보다 크지 않아 모두 실패합니다. 전체 합이 음수가 아니면 마지막 후보는 끝 구간을 통과하고, 앞의 제거된 구간을 감싸 돌아도 부족하지 않습니다.

n=정류장 수. O(n) 시간·O(1) 보조 공간.

134 · 반례와 검증

연료가 가장 많은 곳을 고르는 규칙에는 증명이 없습니다. gas=[5,0,4],cost=[6,1,2]에서는 최대 연료 위치 0이 즉시 실패하고 유일한 시작점 2가 가능합니다.

```
gas=[1,4,2], cost=[3,1,3] → 1
```

실행 전에 상태와 반환값을 적고 위의 경계 사례도 확인합니다.

134 · 구현

```
def gas_start(gas, cost):
    total = tank = start = 0
    for index, (supply, use) in enumerate(zip(gas, cost)):
        total += supply - use
        tank += supply - use
        if tank < 0:
            start, tank = index + 1, 0
    return start if total >= 0 else -1
```

200 · 단계별 힌트

LeetCode 200: Number of Islands · Medium

재풀이

H1. 방문한 육지를 다시 시작점으로 세면 어떻게 되나요?

H2. 대각선도 간선인가요?

H3. 미방문 육지에서 count를 늘리고 연결된 육지를 스택으로 모두 표시합니다.

200 · 정당성과 비용

한 번 시작한 탐색은 연결 경로를 따라 그 성분의 모든 노드와 그 노드만 방문합니다. 이미 표시한 성분은 다시 시작하지 않으므로 시작 횟수가 성분 수입니다.

R,C=행·열 수. O(RC) 시간·O(RC) 보조 공간. 일반 인접 리스트에서는 O(V+E) 시간입니다.

200 · 반례와 검증

값 비교는 ==입니다. is는 객체 동일성입니다. [[“1”,”0”],[“0”,”1”]]은 대각선을 연결하지 않아 두 성분입니다.

```
grid=[["1","0","1"],["1","0","0"]] → 2
```

실행 전에 상태와 반환값을 적고 위의 경계 사례도 확인합니다.

200 · 구현

```
def islands(grid):
    rows, cols = len(grid), len(grid[0])
    seen, count = set(), 0
    for row in range(rows):
        for col in range(cols):
            if grid[row][col] != "1" or (row, col) in seen:
                continue
            count += 1
            stack = [(row, col)]
            seen.add((row, col))
            while stack:
                r, c = stack.pop()
                for nr, nc in ((r-1,c),(r+1,c),(r,c-1),(r,c+1)):
                    if (0 <= nr < rows and 0 <= nc < cols
                            and grid[nr][nc] == "1" and (nr,nc) not in seen):
                        seen.add((nr,nc))
                        stack.append((nr,nc))
    return count
```

실행과 참고자료

```
cd code
python demo_15.py
python demo_16.py
python verify.py
```

표준 라이브러리 import는 `solutions.py`에 있습니다. LeetCode에서는 공식 메서드 서명에 맞춰 함수를 연결합니다.

Erik Demaine and Srini Devadas, Introduction to Algorithms, MIT 6.006 (Fall 2011).

중간 점검의 답과 근거

1. 불가능합니다. 도달 범위의 끝이 2에 머뭅니다.
2. 2. 0→1→4로 이동합니다.
3. 0부터 센 인덱스 1. 순환하며 탱크는 3,2,0입니다.
4. 0. 이미 마지막 위치에 있으므로 추가 점프를 세지 않습니다.

개념 질문 해설

예제 성공은 그리디 규칙의 증명이 아닙니다. 현재 도달 범위의 모든 위치를 확인해야 다음 레벨의 최대 범위를 확정할 수 있습니다.

익숙한 문제를 다시 푼 결과와 바뀐 조건의 문제를 푼 결과는 따로 기록합니다. 혼합 평가에서는 주제 이름보다 입력 계약·상태·정당성에 근거해 풀이를 선택합니다.

순환 경로의 충분조건

차이를 $d[i]=gas[i]-cost[i]$, 접두합을 $P[0]=0$, $P[k]=d[0]+\dots+d[k-1]$로 둡니다. tank가 음수가 될 때 새 출발점으로 옮기는 것은 더 작은 접두합을 만날 때마다 그 위치를 저장하는 것과 같습니다. 최종 출발점 s는 접두합의 최솟값 위치입니다.

s 이후의 누적 연료는 $P[k]-P[s]$이므로 음수가 아닙니다. 앞쪽 k로 돌아온 뒤에는 $P[n]-P[s]+P[k]$입니다. $P[n]\ge 0$이고 $P[k]\ge P[s]$이므로 이 값도 음수가 아닙니다. 따라서 전체 합이 음수가 아니면 최종 후보가 한 바퀴를 완주합니다.